CHALMERS CHALLENGE 2021

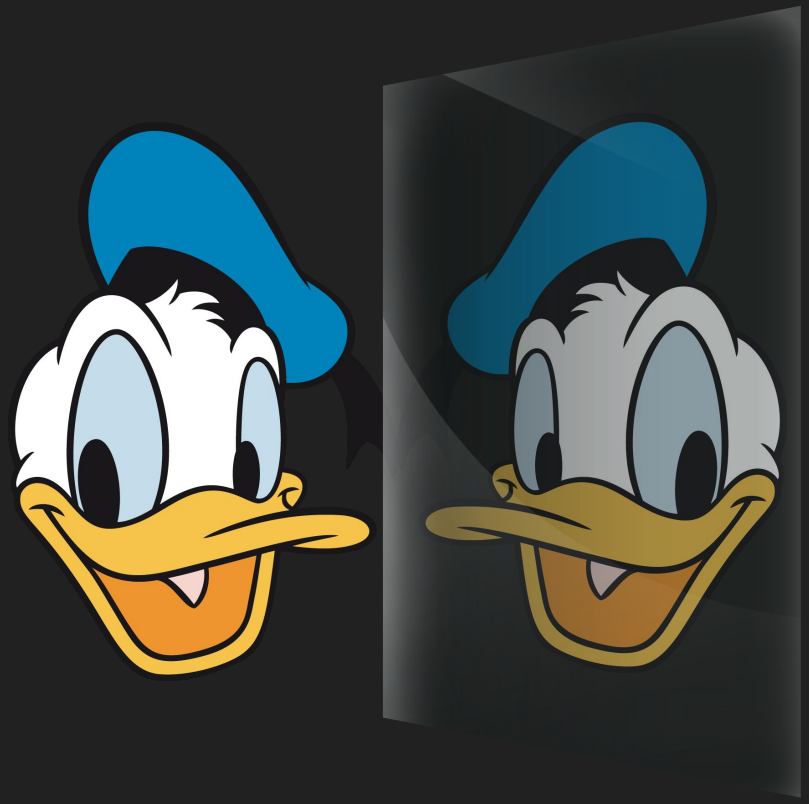
Lösningsgenomgång

av Chalmers Competitive Coding

Chalmers roligaste programmeringstävling!

A: kcuD dlanoD

1) Läs in input som sträng
2) Byt ut alla tvåor mot femmor och alla femmor mot tvåor
3) Spegelvänd talen
4) Skriv 1 om första påsen nu väger mest, annars 2



Lösning av Oskar Veileborg

```python
def f(s):
    l = []
    for c in s:
        if c == '2': l.append('5')
        elif c == '5': l.append('2')
        else: l.append(c)
    return int(''.join(reversed(l)))

N, M = map(f, input().split())

print(1 if N > M else 2)

```

B: CHACTL

1) Beräkna sidskillnaden mellan input

2) Olika angreppssätt:
   a) Rekursivt:
      f(k) är antalet tvåpotenshopp som krävs för att hoppa k sidor.
      $f(0) = 0$
      $f(2k) = f(k)$
      $f(4k + 1) = 1 + f(k)$
      $f(4k + 3) = 1 + f(k+1)$
   b) Greedy
   c) Oeis-formel
   d) Dynamisk Programmering

3) Anropa f för varje sidskillnad och addera

```python
count = int(input())
pages = [int(input()) for i in range(count)]

current_page = 0
target_pointer = 0
jumps = 0

while target_pointer < len(pages):
    target_page = pages[target_pointer]
    if target_page == current_page:
        target_pointer += 1
        continue
    all_jumps = [current_page + 2 ** i for i in range(27)] + [
        current_page - 2 ** i for i in range(27)
    ]
    distances = [abs(pages[target_pointer] - jump) for jump in all_jumps]
    current_page = all_jumps[distances.index(min(distances))]
    if current_page == target_page:
        target_pointer += 1
    jumps += 1

print(jumps)
```

Lösning av Mateusz Drwal

C: Hills in Gothenburg

Hitta kortaste väg: riktad Dijkstra:

- Kostnad från nod i till j är max(0, h(j) - h(i))

Lösning av Jacob Gummesson Atroshi

```python
import heapq as h
def dijkstra(start):
    pq=[(0,start)]
    visited=set([])
    d=[float('inf')]*n
    d[start] = 0
    while pq:
        v=h.heappop(pq)
        node=v[1]
        for dest,cost in G[node]:
            if dest not in visited and d[dest]>v[0]+cost:
                d[dest] = v[0]+cost
                h.heappush(pq,(v[0]+cost,dest))
        visited.add(node)
    return d

n,m = input().split()
n=int(n)

startn, endn = input().split()

G={}
heights = [int(i) for i in input().split()]

for i in range(int(m)):
    i,j = [int(k) - 1 for k in input().split()]
    heightdiff = heights[j] - heights[i]
    if i not in G:
        G[i] = []
    if j not in G:
        G[j] = []
    G[i].append((j,max(0,heightdiff)))
    G[j].append((i,max(0,-1*heightdiff)))

print(dijkstra(int(startn) - 1)[int(endn) - 1])
```

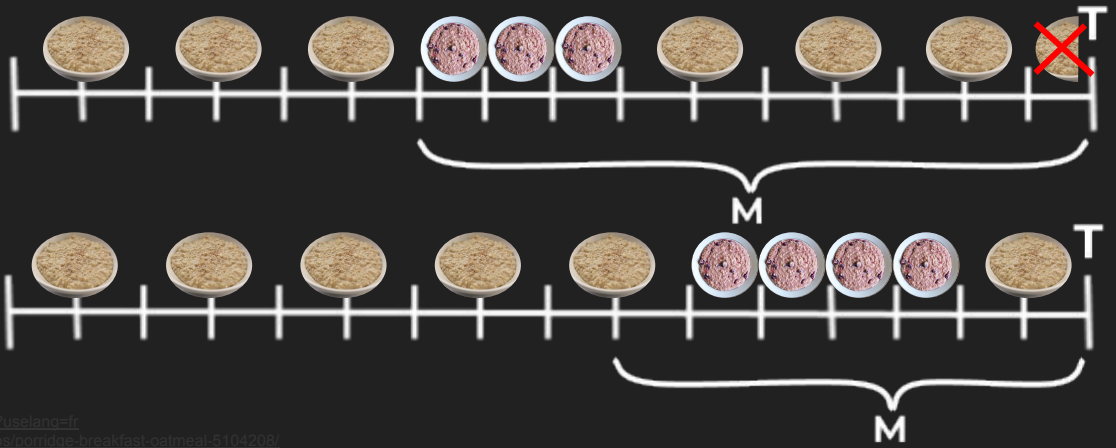
D: Perfect Porridge

1) Sortera otåligheterna

2) Beräkna hur många studenter som blir nöjda för varje otålighet *I*.

T < M + 2

annars:



https://commons.wikimedia.org/wiki/File:Charleroi_-_station_Janson_-_Lucky_Luke_-_01.jpg?uselang=fr
https://pixabay.com/photos/porridge-bowl-of-oats-oatmeal-5496533/ https://pixabay.com/photos/porridge-breakfast-oatmeal-5104208/

```python
N, M = map(int, input().split())

I = sorted(list(map(int, input().split())))

def done_at_time(t):
    lo = 0
    hi = N + 1

    while lo + 1 < hi:
        mid = (lo+hi)//2
        ok = False
        if 2*mid <= M + 1:
            if 2*mid <= t:
                ok = True
            else:
                ok = False
        elif mid > M:
            ok = False
        else:
            reheats = 2*mid - M - 1
            if 2 * mid + reheats <= t:
                ok = True
            else:
                ok = False
        if ok:
            lo = mid
        else:
            hi = mid
    return lo

ans = 0
for i in range(N):
    students_left = N - i
    ans = max(ans, min(students_left, done_at_time(I[i])))

print(ans)
```

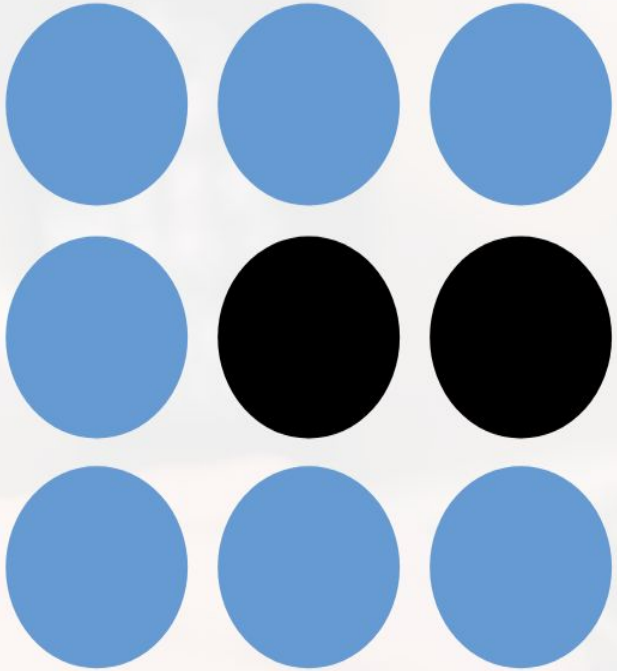
Lösning av Jonatan Nilsson

```python
N, M = [int(q) for q in input().split()]
impatience = [int(q) for q in input().split()]
impatience.sort()
porridges = []

for i in range(N):
    T = impatience[i]
    if T < M + 2:
        porridges.append(min(N-i, T//2))
    else:
        cold = max(0, (T-M)//2)
        extra = (T - M) % 2
        reheats = min(M - extra, cold)
        final_new = max(0, (M - extra - reheats)//2)
        porridges.append(min(N-i, extra + reheats + final_new))

print(max(porridges))
```

E: Antivirus

https://pixy.org/761507/

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<ll, ll> pii;
typedef vector<ll> vi;
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define trav(a, x) for(auto& a : x)
#define all(x) begin(x), end(x)
#define rall(x) rbegin(x), rend(x)
#define sz(x) (int)(x).size()

int main() {
    cin.sync_with_stdio(0); cin.tie(0);
    cin.exceptions(cin.failbit);

    int N; cin >> N;
    vi next(N);
    vi weight(N, 1);
    vi antivirus(N);
    // (antivirus amount, id)
    priority_queue<pair<ll,int>> biggest_antivirus;
    priority_queue<pair<ll,int>> biggest_virus;
    ll day = 0;

    rep(i, 0, N) {
        cin >> next[i]; next[i]--;
    }
    rep(i, 0, N) {
        cin >> antivirus[i];
        if (antivirus[i] > 0) {
            biggest_antivirus.push(make_pair(antivirus[i], -i));
        } else if (antivirus[i] < 0) {
            biggest_virus.push(make_pair(-antivirus[i], -i));
        }
    }
    // While there exist viruses
    while (!biggest_virus.empty() && biggest_virus.top().first > 0) {
        if (-biggest_virus.top().first != antivirus[-biggest_virus.top().second]) {
            biggest_virus.pop();
            continue; // Repeat if we got invalidated virus pile
        }
        if (biggest_antivirus.empty()) {
            cout << "never\n";
            return 0;
        }

        ll pop = biggest_antivirus.top().first;
        int id = -biggest_antivirus.top().second; // stored negated
        biggest_antivirus.pop();
        if (antivirus[id] != pop) {
            continue; // Repeat if we got invalidated antivirus pile
        }

        int child = next[id];

        // Bail if we are pushing a non-positive or pushing around in a circle
        if (pop < 1 || (antivirus[child] == 0 && child == 0 && next[child] == id)) {
            cout << "never\n";
            return 0;
        }

        // Contract edge between child and next[child] if that's valid
        if (antivirus[child] == 0 && child != 0 && child != next[0]) {
            next[id] = next[child];
            weight[id] += weight[child];
            child = next[id]; // requires this new value now
        }

        //cout << "moved " << pop << " from node " << id << " to node " << child << e
        day += weight[id];
        antivirus[child] += antivirus[id];
        antivirus[id] = 0;

        // id is not placed in a queue as it has value 0
        if (antivirus[child] > 0) {
            biggest_antivirus.push(make_pair(antivirus[child], -child));
        } else if (antivirus[child] < 0) {
            biggest_virus.push(make_pair(-antivirus[child], -child));
        }
    }
    cout << day << endl;
}
```



CHALMERS COMPETITIVE CODING